XenSense-V1

Real-time video segmentation & scene understanding for autonomous driving

Part 1 of 2 — **Capabilities & Results Walkthrough**

Every module in the perception pipeline, shown on real annotated frames.

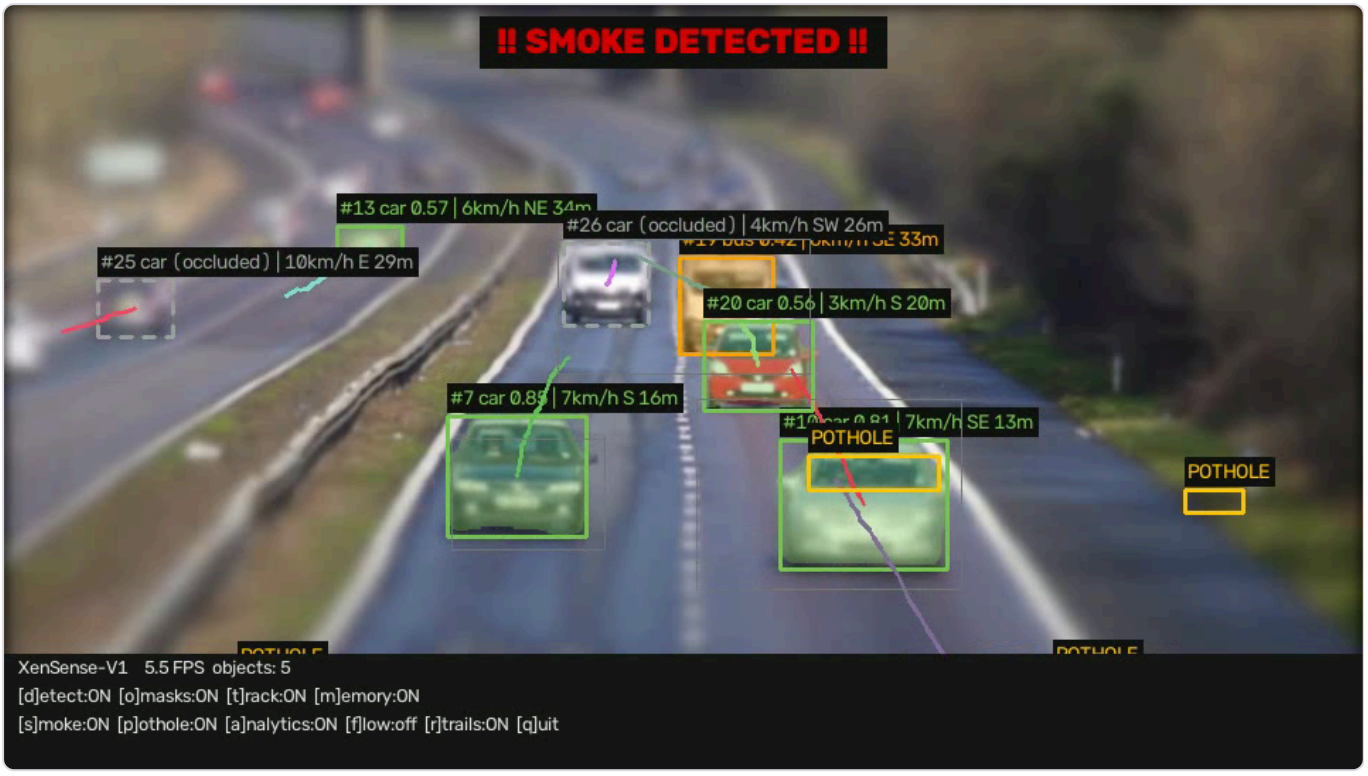
Python · PyTorch · Ultralytics YOLOv8 · OpenCV · DeepSORT

Prepared 14 July 2026 · Author: Bhanu Prakash

0 · What XenSense-V1 is

XenSense-V1 is a modular, real-time **video perception pipeline for autonomous-driving systems**. It ingests a driving video (or webcam feed) and produces an annotated stream plus CSV/JSON logs — unifying instance segmentation, multi-object tracking, occlusion-aware memory, hazard detection (smoke/fog and potholes), and motion analytics into one system where every capability can be toggled live. It is the code implementation of a research report (BMS College of Engineering, VTU) that ships as `REPORT.pdf`.

> **The interviewer one-liner:** "It's a single-camera ADAS perception stack — YOLOv8 segments and labels every road object, DeepSORT keeps stable IDs through occlusion with a lightweight memory bank, and classical + CNN modules add smoke/pothole hazard flags and per-object speed, direction and distance — all rendered live with a togglable HUD and exported to CSV/JSON."



The flagship output frame — one glance shows the whole system working: the red "SMOKE DETECTED" banner, per-object boxes labelled with id/class/confidence/speed/direction/distance (e.g. "#7 car 0.85 | 7km/h S 16m"), dashed "occluded" ghost boxes, POTHOLE boxes, coloured trajectory trails, and the bottom HUD with live FPS and every module's ON/OFF state.

Stack & models

COMPONENT	MODEL / LIBRARY	ROLE
Segmentation	YOLOv8n-seg (Ultralytics)	Boxes, class labels, confidence, pixel instance masks
Tracking	DeepSORT + IoU fallback	Persistent track IDs across frames
Hazard (smoke)	Custom SmokeCNN *or* classical CV	Smoke/fog detection with temporal voting
Depth priors	Class-based real-world size priors	Monocular distance estimation (metres)
Backbone	PyTorch (CPU or CUDA)	Runs YOLOv8 + SmokeCNN; GPU auto-used if present

Honest engineering note: the `weights/` folder ships no trained smoke/pothole weights, so those two modules currently run through their **classical-vision fallbacks** (still fully functional). Speed and direction are **camera-relative** — there is no ego-motion compensation yet, so on a moving dashcam an object matching the car's own speed reads near 0 km/h. Both are documented as future work.

1 · The pipeline, capability by capability

Instance segmentation & semantic labelling

YOLOv8-seg produces a bounding box, class label, confidence and a pixel-level instance mask for every detection, filtered to driving-relevant COCO classes (person, bicycle, car, motorcycle, bus, truck, traffic light, stop sign, train, dog, cow) — or all 80 classes with `--all-classes`. The coloured mask overlays and class labels in the hero frame come from this stage.

Multi-object tracking

DeepSORT (cosine-distance appearance embeddings + Kalman filtering) assigns each object a persistent track ID so it can be followed across frames. A dependency-free IoU tracker is the graceful fallback. The `#7`, `#13`, `#25` ids on the boxes are these track IDs.

Occlusion-aware memory recall

A highway frame — tracked vehicles with trajectory trails and per-object analytics.

A compact per-track memory (last box + smoothed velocity + HSV colour histogram) predicts "ghost" boxes for objects that become occluded (drawn dashed), and re-identifies them on reappearance via histogram matching — so an object that disappears behind another keeps its ID and trajectory instead of being counted as new. This is the STM/SRNet-inspired part, kept lightweight.

Smoke / fog hazard detection

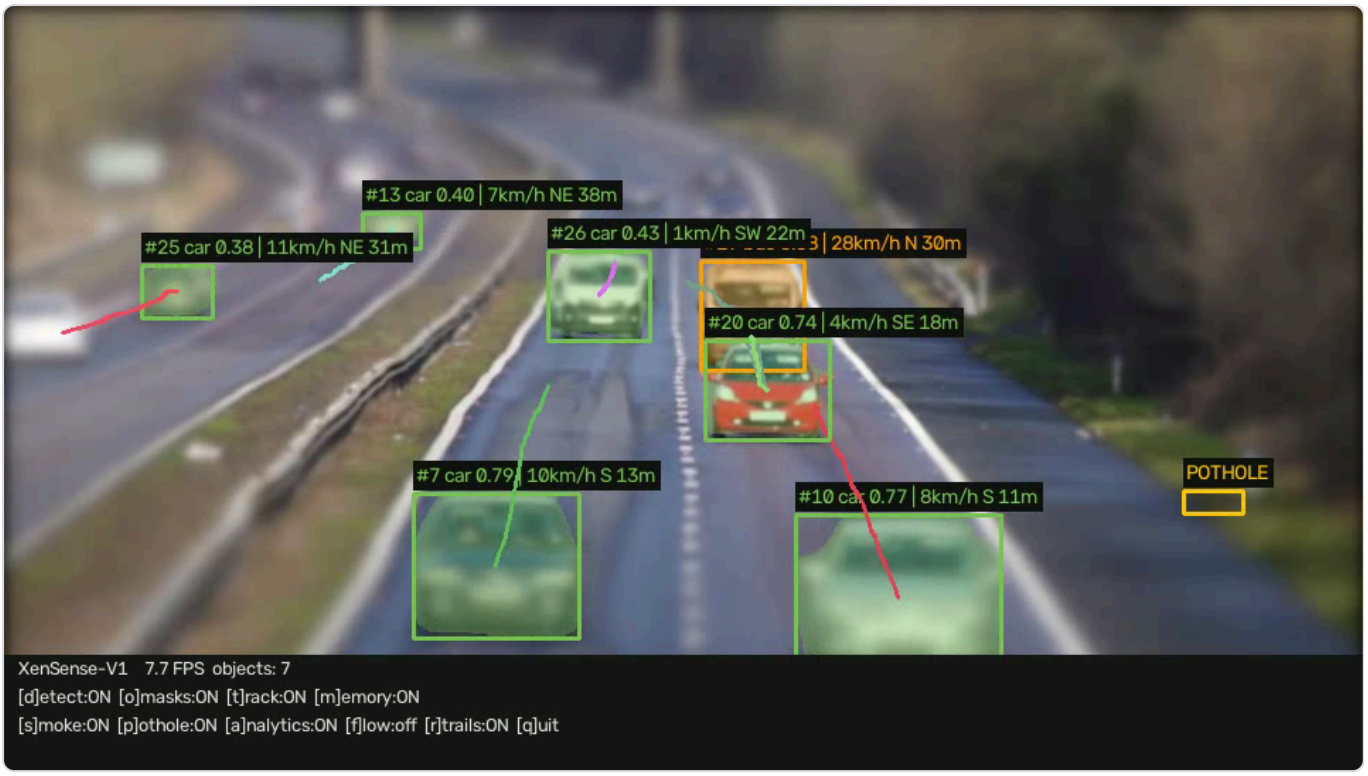
The synthetic test clip, which deliberately exercises the smoke and pothole heuristics.

Dual backend: the trained SmokeCNN when weights are present, else a classical colour + temporal-drift + texture heuristic that localises candidate smoke regions. A temporal majority-vote filter suppresses false positives, a global fog cue flags bright low-contrast frames, and cross-module fusion excludes detected-object motion. When triggered, the red hazard banner appears (as in the hero frame).

Pothole / surface-anomaly detection

Dual backend again: a custom pothole YOLO when present, else a classical dark-elliptical-blob detector confined to the road ROI, with shadow suppression under vehicles and temporal-persistence validation (a pothole must appear in ≥3 of the last 5 frames). The yellow POTHOLE boxes come from this stage.

Object analytics — speed, direction, distance



Per-object labels carry speed (km/h), 8-way compass direction, and estimated distance in metres.

For each tracked object the system computes speed in km/h (from pixel + optional depth displacement), an 8-way compass direction, and a monocular distance estimate using class-based real-world size priors (metres) against a focal-length prior — the "`7km/h S 16m`" style labels. A centroid trail records recent trajectory.

Optical flow, rendering & export

- **Optical flow** — dense Farnebäck flow on a downscaled frame, rendered as sparse global-motion arrows (off by default).
- **Rendering + HUD** — composites masks, boxes, IDs, trails, hazard banners, ghost boxes and a status HUD with EMA-smoothed live FPS.
- **Data export** — per-frame/per-object CSV rows plus a JSON summary (class counts, unique-object counts, hazard event ranges, pothole frame counts, average FPS).
- **Runtime modularity** — every module toggles live from the keyboard, and via CLI start-state flags.

> **Measured results (from the run logs):** real highway clip — 120 frames at ~6.2 FPS on CPU, **766 car detections across 17 unique vehicles** + 1 bus, two smoke events (frames 11–32 and 73–90), potholes in 118 frames. Synthetic clip — 200 frames at **~24 FPS**, two smoke events, potholes in 198 frames. The underlying methodology and evaluation are written up in `REPORT.pdf`.

2 · Likely interview questions

Q: How does the occlusion memory preserve object identity?
Each track stores its last box, a smoothed velocity, and an HSV colour histogram. When a detection drops out, the memory predicts a "ghost" box from that velocity for up to N frames; when a detection reappears nearby, a histogram match re-attaches the old ID instead of minting a new one — so trajectories survive short occlusions.

Q: How do you estimate distance from a single camera?
Monocular geometry with priors: each class has a known real-world size (a car is ~1.8 m wide, etc.). Comparing that prior to the object's pixel size and a focal-length prior yields an approximate distance in metres. It's an estimate, not LiDAR, but good enough for near/approaching-threat filtering.

Q: Why a CNN and a classical fallback for smoke?
The CNN is more accurate when trained weights exist; the classical colour/texture/temporal-drift heuristic needs no weights and still works out of the box. Shipping both means the feature degrades gracefully rather than failing when weights are absent — which is the current state of this repo.

Q: How is it real-time on modest hardware?
Detection and depth run every N frames (not every frame) with cached results in between, input is resized to a processing width, and FPS is EMA-smoothed. It targets CPU by default and auto-uses CUDA when available.

Q: What's the biggest known limitation?
Speed/direction are camera-relative — no ego-motion compensation — so a dashcam moving at the same speed as a tracked car reads it as ~stationary. Adding ego-motion correction is the top future-work item.